

Práctica 2: Reconocimiento de texto

El enunciado corresponde a la segunda práctica calificable: **entrega día del examen de mayo.**

Importante: En esta práctica se podrán utilizar las técnicas de procesamiento de imagen así como técnicas sencillas de aprendizaje automático: scikit-learn, y módulos ml de OpenCV (no se puede utilizar Deep Learning dado que no es necesario).

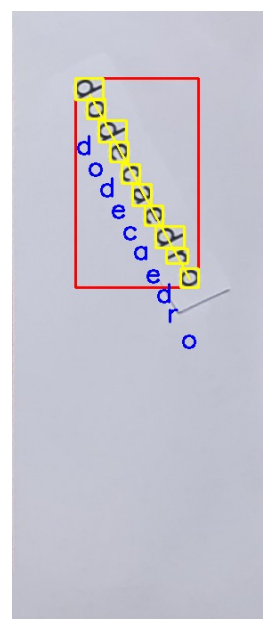


Figura 1: Ejemplo de imagen con la plantilla, el nombre del modelo 3D y el resultado esperado (izquierda). A la derecha podemos ver la imagen rectificada y el resultado del OCR.

1 Copias de código o de la memoria

El código desarrollado en las prácticas debe de ser original. La copia (total o parcial) de prácticas será sancionada, al menos, con el SUSPENSO en la misma. En estos casos, podrá significar, en la siguiente convocatoria y a discreción del profesor, el tener que **resolver nuevas pruebas y la defensa de las mismas de forma oral. Las sanciones derivadas de la copia, afectarán tanto al alumno que copia como al alumno copiado.**

Para evitar que **cuando se usa código de terceros** sea considerado una copia, se debe **citar siempre la procedencia** de cada parte de **código no desarrollada por el propio alumno** (con comentarios en el propio código y con mención expresa en la memoria de las prácticas). El plagio o copia de terceros sin la cita correspondiente (p.ej. una página web o a través de ChatGPT o similares) ya sea

en el código a desarrollar en las prácticas y/o de parte de la memoria de las prácticas, acarrearán las mismas sanciones que en la copia prácticas de otros alumnos.

2 Reconocimiento Óptico de Caracteres (OCR)

En esta práctica se desea construir un sistema que permita la localización y lectura automática del texto que aparecerá en la parte blanca de la hoja impresa con la plantilla utilizada en la Práctica 1 (ver Figura 1). Partiremos de la detección de la plantilla realizada en la Práctica 1. En esta práctica implementaremos los pasos la lectura automática del texto (reconocimiento de los caracteres) que identificará el modelo 3D a proyectar sobre la imagen de entrada de entre los que se encuentran en un directorio con los modelos posibles.

Para resolver el problema tendremos que:

- **Entrenar y validar un clasificador de caracteres.** Se utilizarán las imágenes provistas en el directorio *train_ocr* (ver Figura 2) para entrenar un clasificador multiclase, con una clase por cada tipo de carácter (letra o número). En las imágenes tenemos cada uno de los caracteres que podemos ver con algunas transformaciones geométricas y añadido algo de ruido lo que proporciona variedad a los ejemplos que verá el clasificador.



Figura 2: Ejemplos de imágenes de entrenamiento para el clasificador de texto

- **Detectar los caracteres alineados.** En este caso se tratará de detectar los caracteres y las líneas de texto que forman (p.ej. con RANSAC de líneas). En la Figura 1 se muestra una posible detección de caracteres y líneas de texto.
- **Lectura del texto del panel (OCR).** Para la lectura automática procederemos a ejecutar el clasificador de caracteres entrenado siguiendo las líneas de texto.

2.1 Ejercicio 1: Entrenamiento y validación del clasificador de caracteres

Para construir el sistema de clasificación se proporciona un conjunto de entrenamiento que contiene variaciones de cada uno de los caracteres que puede aparecer al lado de la plantilla (los caracteres: 0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ).

En el Aula Virtual se proporciona un esquema de código para facilitar la implementación de las pruebas a realizar. En particular se propone (aunque no es obligatorio) utilizar una jerarquía con una clase base *OCRClassifier* y diferentes clasificadores como clases derivadas que implementan los métodos *fit* y *predict*.

Para entrenar el clasificador básico (el método *fit*) se pide seguir los siguientes pasos:

- Cargar y umbralizar las imágenes de los caracteres de entrenamiento (por ejemplo, `cv2.adaptiveThreshold + cv2.findContours + cv2.boundingRect` o directamente `MSER + cv2.boundingRect`). Este proceso habrá que hacerlo también en las imágenes con líneas de texto. De esta manera el clasificador verá el mismo tipo de imágenes en entrenamiento y pruebas reales.
- Construir el vector de características a partir de las imágenes de entrenamiento del carácter ya recortadas usando el rectángulo detectado. En la versión básica a implementar se sugiere utilizar directamente los niveles de gris. Todos los vectores de características deben de ser del mismo tamaño con lo que habrá que redimensionar la imagen del carácter a ese tamaño fijo (por ejemplo a 30x30 píxeles). Convertir la matriz con los niveles de gris del carácter ya redimensionada, con un tamaño de 30x30 y valores entre 0 y 255, en una matriz 1x900. Cada una de estas matrices filas se usará como vector de características asociado al carácter de entrenamiento.
- **Opcionalmente**, reducir la dimensión de los vectores de características (utilizando LDA, PCA u otros métodos en `skit-learn`):
- **Entrenar un clasificador** para usarlo para reconocer los caracteres individualmente. En la biblioteca *scikit-learn* y también en la biblioteca OpenCV (en el módulo `ml` de “machine learning”) tenemos implementados los clasificadores vistos en la asignatura de Aprendizaje Automático I y II (*Ensemble classifiers*). En la práctica se trata de probar los diferentes clasificadores y escoger el que obtenga el mejor resultado. Si se utilizan los clasificadores de OpenCV, habrá que tener en cuenta que los métodos en el módulo *ml* únicamente aceptan matrices de tipo `np.float32`, y no `np.float64` como utiliza por defecto *scikit-learn*. Habrá que hacer una conversión de tipos antes de pasar matrices de una a otra bibliotecas. Además, el vector de etiquetas en OpenCV tiene que ser de tipo `np.int32` para que no se produzca un error de ejecución.
- En este primer ejercicio **se pide evaluar el clasificador sobre el conjunto de imágenes en el directorio de `test_ocr_char`**. Se deberán utilizar cuantas medidas de rendimiento del clasificador se crean oportunas (lo que redundará en obtener la máxima nota) aunque el mínimo será la tasa de acierto (`accuracy`) y el `f1-score macro`.

La implementación de este ejercicio se hará completando el script proporcionado `main_prac2_train_ocr.py`. A este se le pasará el path al directorio `train_ocr`, al directorio `test_ocr_char` y el nombre del clasificador que se quiera probar (internamente en el script el alumno permitirá elegir el clasificador a probar con este parámetro).

Una de las características que permitirá acelerar el desarrollo de las pruebas será el poder **guardar el clasificador entrenado en disco**. En python se puede guardar un objeto en disco con `pickle` y recuperarlo en el estado que se guardó. **Desafortunadamente si se usan clasificadores del módulo `ml` de OpenCV no se podrá usar `pickle` directamente, mientras que con `skit-learn` sí.**

2.2 Ejercicio 2. Probar otras alternativas para el reconocimiento.

El ejercicio 2 consiste en comparar los resultados que ofrece el sistema de reconocimiento con otras alternativas. Entre las posibles alternativas se proponen:

- Utilizar otros vectores de características (a parte de los niveles de gris) en el clasificador básico:
 - Otras medidas disponibles en OpenCV o en `skimage.feature` (<http://scikit-image.org/docs/dev/api/skimage.feature.html>), descriptores de características (p.ej. SIFT), etc.
- Otros algoritmos de reducción de dimensionalidad¹:
 - Por ejemplo, PCA (Principal Component Analysis) disponible en `sklearn` (<http://scikit-learn.org/stable/modules/decomposition.html>) o también la ausencia de reducción de dimensionalidad.
- Utilizar otros clasificadores:
 - KNN, Bayesiano con Gaussianas, Random Forest o Randomized Trees, o cualquier otro clasificador (supervisado) disponible en OpenCV y/o `sklearn`. Es importante hacer notar aquí que el clasificador tiene que ser multi-clase.

El código desarrollado para esta parte de la práctica deberá permitir elegir cada uno de los diferentes clasificadores mediante el parámetro correspondiente en `main_prac2_train_ocr.py`. Además, la comparativa entre el sistema básico y las alternativas implementada en esta sección quedará reflejada en la memoria de la práctica. La claridad y adecuación en la exposición de las medidas de rendimiento (tasas de acierto, matrices de confusión, gráficos, curvas ROC o de Precisión/Recall, etc.) será valorada en la nota.

Para obtener la **máxima puntuación** habría que probar con relativo éxito sobre el conjunto de datos del directorio `test_ocr_char`, **2 alternativas al sistema básico del Ejercicio 1** (entendiendo como alternativa el cambio de cualquiera de los componentes del sistema básico – extracción de características, algoritmo de reducción de dimensionalidad o clasificador) con una discusión adecuada de los resultados.

2.3 Ejercicio 3: Lectura automática de texto

En este caso se trata de poder localizar los caracteres y la línea de texto en imágenes en las que aparece una única línea. Las imágenes de test se encuentran en el directorio `test_ocr_words_plain` (ver Figura 3). Adicionalmente, en este ejercicio se pretende que el alumno aplique el mejor clasificador del apartado anterior a los caracteres localizados en la imagen (ver Figura 3, derecha).

1. Al usar LDA o PCA como reducción de dimensionalidad con otros clasificadores (distinto del que proporciona la clase LDA de `sklearn`), implicará que hay que usar el método `transform` para bajar la dimensionalidad antes de entrenar y también al ejecutar los clasificadores.



Figura 3: Ejemplo de imagen de una imagen con una línea de texto (izquierda), y detección de caracteres, líneas de texto con RANSAC y estimación de la clase de cada carácter (derecha).

Un posible algoritmo para este ejercicio sería:

1. Detectar los caracteres mediante umbralización y búsqueda de componentes conexas (MSER o utilización de `cv2.adaptiveThreshold + cv2.findContours`). Se deberán eliminar aquellos rectángulos con relaciones de aspecto no comparables a las de un carácter.
2. Encontrar los caracteres alineados mediante, por ejemplo, RANSAC sobre los centros de cada carácter.
3. Se ejecutará el clasificador sobre cada uno de los caracteres en el orden de la línea (orden de lectura). Se generará un único string de salida que representará la línea de texto.

Todas las prácticas en este apartado se implementarán en un script `main_prac2_train_ocr.py` del que se proporciona un esquema en el Aula Virtual. La salida de este script será un fichero `result_text_lines.txt`. El fichero de texto con las anotaciones, una por línea, sigue el siguiente formato:

```
<nombre_fichero>;<x1>;<y1>;<x2>;<y2>;<texto_ocr>
```

En los parámetros `x1`, `y1`, `x2`, `y2` se colocarán los de una ventana que ocupa toda la imagen (al fin y al cabo la imagen completa es un panel recortado). En `<texto_ocr>` deberemos escribir el string de salida del panel con el formato explicado anteriormente.

Se proporciona un script, `evaluar_resultados_test_ocr.py`, que permite comparar el resultado de nuestro fichero de texto con el resultado de la práctica (ver Figura 4) con el fichero con las anotaciones hechas a mano (`gt.txt` en `test_ocr_words_plain`). Este script nos dará una gráfica con el histograma que compara el texto del texto reconocido por nuestra práctica con las correctas utilizando la distancia de Levenshtein (número de cambios en una cadena de texto para convertirla en otra).

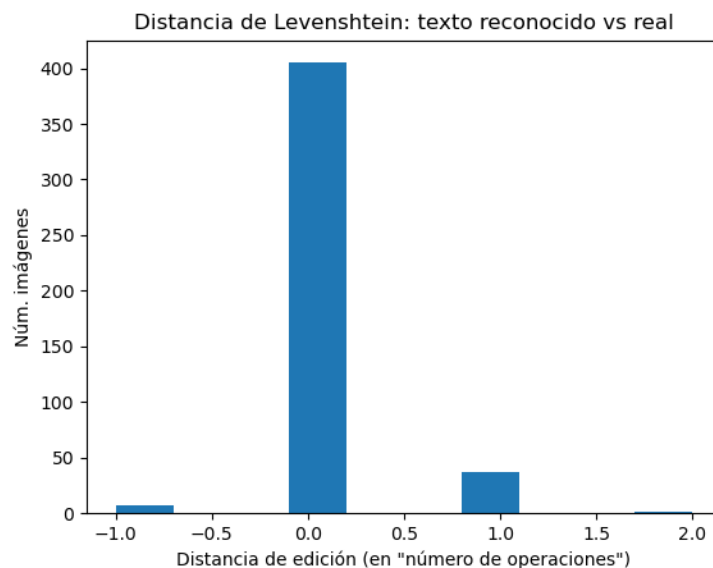


Figura 4: Resultados de ejemplo para el directorio `test_ocr_words_plain`. El histograma nos dice que hay unas pocas imágenes para las que no se ha podido encontrar el texto (distancia -1), >400 imágenes con distancia de Levenshtein 0 y unos pocos con errores en 1 carácter.

2.4 Ejercicio 4: Lectura del nombre del modelo 3D y visualización

El ejercicio 4 consiste en unir el sistema de reconocimiento diseñado en el apartado anterior con el detector de la plantilla en 3D desarrollado en la práctica 1. En el script `main_prac2.py` copiaremos el `main.py` de la práctica 1 y añadiremos la lectura del texto que aparece en la parte blanca de la hoja DIN A4 en la que está impresa la plantilla (a su derecha). Para ello habrá que realizar los siguientes pasos:

1. Cargar los modelos 3D que se encuentran disponibles en el directorio `3d_models` del material. Se recomienda usar un diccionario para almacenarlos siendo la clave el nombre del fichero sin su extensión pasado a mayúsculas (`cubo.obj` → `clave = CUBO`)
2. Detectar la plantilla tal y como se hizo en la práctica 1. Esto nos permitirá obtener $\mathbf{H}_t^{\text{img}}$.
3. Utilizando la homografía $\mathbf{H}_t^{\text{img}}$ y `cv2.warpPerspective`, rectificar la parte donde aparece el texto de en la hoja (ver parte derecha de la Figura 1).
4. Aplicar el OCR desarrollado en el Ejercicio 3 a la imagen rectificada y obtener el texto correspondiente. Pasarlo a mayúsculas y encontrar el modelo 3D que más se parezca utilizando la distancia de Levenshtein. Esto último permitirá algunos errores de reconocimiento y aún así seleccionar el modelo correcto.
5. Dibujar los ejes del sistema de referencia sobre el plano y el modelo 3D seleccionado en el mismo sitio que en la Práctica 1 y con la misma escala.

Durante la ejecución del script **se mostrará en pantalla, al menos** (se puede decidir enseñar más información): la imagen que se está procesando, el sistema de referencia sobre el plano 3D, el objeto 3D proyectado sobre la imagen, la imagen de la zona de texto rectificada y el resultado del OCR sobre la misma (incluyendo la detección de caracteres y la línea que los une). La visualización será de tal manera que el usuario tenga que dar a una tecla para pasar a la siguiente imagen.

3 Datos de entrenamiento, test y formato de salida

Los **datos de entrenamiento del OCR** proporcionados son imágenes .png con caracteres (directorio *train_ocr*).

Los **datos de test del OCR** proporcionados son imágenes .png con caracteres (directorio *test_ocr_char*).

Las **imágenes de test de la lectura automática de líneas de texto** se encuentra en el directorio *test_ocr_words_plain* contiene imágenes de una única línea de texto en negro sobre fondo blanco. Para **estas imágenes** se proporciona también un fichero de anotaciones *gt.txt* para poder establecer la distribución de errores en el texto reconocido.

Todas las prácticas entregadas deberán implementar los ejercicios de la siguiente manera:

- Ejercicios 1, 2 y 3 en el script *main_prac2_train_ocr.py*
- Ejercicio 4 en el script *main_prac2.py*

Se deberán utilizar los scripts python *evaluar_resultados_test_ocr.py*, que se proporcionan junto con este enunciado, en los ejercicios 3 y 4. Los resultados se discutirán en la memoria.

Para el **Ejercicio 4** se copiará el script *main.py* de la práctica 1 en *main_prac2.py* incluyendo la ejecución del OCR y la selección del modelo 3D a pintar con el texto y el guardado del texto reconocido en el fichero *results_text_lines.txt*.

4 Normas de presentación

La presentación seguirá las siguientes normas:

- Su presentación se realizará a través del Aula Virtual.
- Para presentarla se deberá entregar un único fichero ZIP que contendrá el código fuente en python y una memoria en formato PDF con la descripción del sistema desarrollado.
- En la memoria se incluirá una explicación con el algoritmo desarrollado, los métodos de OpenCV y/o sklearn utilizados, copias de las pantallas correspondientes a la ejecución del programa y unas estadísticas correspondientes al resultado de la ejecución del programa sobre las muestras de test. Si se han probado diferentes sistemas de clasificación deberán reflejarse los diferentes resultados que se hayan obtenido.
- Se permitirán grupos de 3 alumnos.

La puntuación de esta práctica corresponde al 30 % de la asignatura. La práctica se valorará sobre 10 y cada parte tendrá la siguiente puntuación:

- Ejercicio 1: **4 puntos.**
- Ejercicio 2: **2 puntos.**
- Ejercicio 3: **2 puntos.**
- Ejercicio 4: **2 puntos.**

Para obtener la máxima nota en cada apartado, se valorará:

- Implementar los scripts como se pide en los diferentes ejercicios.
- La limpieza y organización del código en clases con un Diseño Orientado a Objetos razonable.
- El funcionamiento del código en las imágenes de test.
- *Las ideas propias o técnicas pensadas por los alumnos para mejorar lo que se propone en el enunciado.*